

1. Effizienz von Algorithmen

1.1. Asymptotische Notation

$g = O(f)$	g wächst maximal so schnell wie f $0 \leq \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $O(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \Omega(f)$	g wächst mindestens so schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq \infty$ $\Omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$
$g = \Theta(f)$	g wächst genauso schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $\Theta(f) = O(f) \cap \Omega(f)$ $c_1 f(n) \leq g(n) \leq c_2 f(n)$
$g = o(f)$	g wächst langsamer als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$ $o(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \omega(f)$	g wächst schneller als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \infty$ $\omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$

1.1.1 Rechenregeln

Für O, Ω, Θ :

- $cf(n) = O(f(n))$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$
- $O(f(n))O(g(n)) = O(f(n)g(n))$
- $\log n < \sqrt{n} < n < n \log n < n^{3/2} < n^2 < n^3 < 1.1^n < 2^n < n!$

1.1.2 Summenregeln

- $\sum_{i=1}^n c = cn$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
- $\sum_{i=0}^n a^i = \frac{a^{n+1}-1}{a-1}$

2. Wichtige Algorithmen

2.1. Sortieralgorithmen

2.1.1 InsertionSort

1. Wähle num = input[i]
2. Vergleiche alle Indices j < i und schiebe num nach input[j], wenn input[j] < num

Laufzeit	Best case: $O(n)$ falls Input sortiert
	Worst case: $O(n^2)$ falls Input absteigend sortiert
	$O(hn)$ falls ein Element höchstens h Elemente von seiner Zielposition entfernt ist

2.2. Suchalgorithmen

2.2.1 Binary Search

Annahme: Liste sortiert, gesucht x .

1. Vergleiche x mit dem Wert, der in der Mitte steht
 - a) Bei Gleichheit sind wir fertig
 - b) Falls x kleiner als der mittlere Wert, gehe zu 1. mit dem linken Teil des Arrays
 - c) Falls x größer als der mittlere Wert, gehe zu 1. mit dem rechten Teil des Arrays

Laufzeit	Best case: $O(1)$ falls mittlere Wert gleich gesuchter Wert
	Worst case: $O(\log_2 n)$

3. Datenstrukturen

3.1. Linked List